

# Appunti — Modulo 3C: Gestione dei Processi

**Corso:** Lab Amministrazione di Sistemi T **Stato:** □ Es. 1-4 eseguiti · Es. 5 da fare · watch da correggere (vedi Es. 3)

---

## 1. Processo: cos'è e come si identifica

Ogni comando lanciato dalla shell diventa un **processo**. Ogni processo ha:

- **PID** (Process ID): numero univoco globale nel sistema, assegnato dal kernel.
- **Job ID**: numero locale alla shell corrente, usato solo nella sessione. Si riferenzia con %, es. %1.

Un processo agisce **a nome dell'utente che lo ha lanciato** — i permessi del processo sono quelli dell'utente. Solo i processi root possono declassarsi a un altro utente, senza possibilità di tornare indietro.

**Come nasce un processo** — la shell chiama: 1. `fork()` — crea una copia del processo corrente (stesso codice, stesso spazio di memoria) 2. `exec()` — sostituisce il codice in memoria con quello del programma da eseguire

**fork()/exec() spiegati:** `fork()` non crea il nuovo programma direttamente, crea una copia identica del processo shell. Poi il figlio chiama `exec()`, che *sovrascrive* il proprio codice con quello del programma richiesto (`sleep`, `ls`, ecc.). Il risultato è un processo con un'identità nuova ma che ha ereditato dal padre file descriptor, variabili d'ambiente e altre risorse. `fork()` senza `exec()` è il pattern dei demoni che si dividono in padre/figlio.

Ogni processo figlio **eredita i file descriptor aperti dal padre**.

**Cosa sono i file descriptor:** il kernel rappresenta ogni risorsa aperta (file, pipe, socket) con un numero intero detto file descriptor (fd). I tre standard sono sempre presenti: 0 = stdin, 1 = stdout, 2 = stderr. Quando fai `echo "ciao" > file.txt`, la shell apre `file.txt` e ottiene un fd, poi passa quell'fd al processo figlio. Ogni processo figlio eredita tutti gli fd aperti del padre — ecco perché il redirect impostato prima del `&` funziona anche per il processo in background.

---

## 2. Processi in foreground e background

Per default un comando è in **foreground**: la shell aspetta la sua terminazione prima di mostrare il prompt.

Per lanciarlo in **background** si aggiunge &:

```
sleep 1000 &  
# [1] 12516 ← job id + PID
```

**Variabile speciale \$!:** contiene il PID dell'ultimo processo lanciato in background.

```
sleep 1000 &  
A=$!  
echo $A      # 12516
```

**\$! vale solo per background:** sì, \$! è definito solo dopo un &. Per processi in foreground, il PID è disponibile solo dentro il processo stesso (come \$). Non esiste una variabile bash che cattura automaticamente il PID di un foreground process.

Un processo in background: - Non riceve input dal terminale - Continua a scrivere su STDOUT e STDERR (l'output appare a schermo, si può redirigere)

**Da foreground a background** — passaggio che non era chiaro:

```
sleep 1000          # lanciato in foreground, shell bloccata  
# premi Ctrl+Z → invia SIGTSTP → processo si SOSPENDE (stato stopped)  
bg %1              # invia SIGCONT → riprende in background
```

Ctrl+Z non uccide il processo, lo sospende. bg lo sblocca e lo manda in background. fg invece lo riporta in foreground.

---

### 3. Segnali

I segnali sono **eventi asincroni** notificati dal kernel a un processo.

**Asincrono = cosa significa:** il segnale può arrivare in *qualunque momento* durante l'esecuzione del processo, indipendentemente da cosa stia facendo (potrebbe essere nel mezzo di un calcolo, in attesa di I/O, ecc.). Il processo non deve fare una chiamata esplicita per riceverlo — il kernel lo inietta nel flusso di esecuzione alla prima occasione utile.

**Cosa può fare un processo alla ricezione di un segnale:** - **Terminare** (con o senza core dump) - **Ignorarlo** - **Sospendersi** (stato stopped) - **Riprendersi** dallo stop (CONT)

**Core dump:** quando un processo termina per certi segnali (es. SIGSEGV, SIGQUIT), il kernel può scrivere su disco una copia dello stato di memoria del processo al momento della terminazione (il

“core file”). Serve per debugging post-mortem: con `gdb ./programma core` puoi vedere esattamente dove si trovava il processo quando è andato in crash.

**CONT:** SIGCONT (continue) sblocca un processo che era in stato stopped. Il processo riprende esattamente da dove era stato sospeso — non ricomincia da capo.

### Segnali importanti:

| Segnale | Numero | Scorciatoia | Effetto default                             |
|---------|--------|-------------|---------------------------------------------|
| SIGTERM | 15     | —           | Termina (graceful, intercettabile)          |
| SIGKILL | 9      | —           | Termina (forza, <b>non intercettabile</b> ) |
| SIGINT  | 2      | Ctrl+C      | Interrompe il processo                      |
| SIGQUIT | 3      | Ctrl+\      | Termina con core dump                       |
| SIGTSTP | 20     | Ctrl+Z      | Sospende (intercettabile)                   |
| SIGSTOP | 19     | —           | Sospende ( <b>non intercettabile</b> )      |
| SIGCONT | 18     | —           | Riprende un processo sospeso                |
| SIGHUP  | 1      | —           | Hangup (chiusura terminale)                 |

### Invio segnali con kill:

```
kill <PID>           # invia SIGTERM (default)
kill -9 <PID>         # invia SIGKILL
kill -STOP <PID>      # sospende
kill -CONT <PID>      # riprende
kill -l               # elenca tutti i segnali
kill -<N> <PID>       # segnale per numero
kill -9 -<PGID>       # SIGKILL a tutto il process group
```

**kill non solo per uccidere:** il nome è fuorviante. `kill` invia *qualsiasi* segnale — non solo SIGTERM o SIGKILL. È lo strumento generico per parlare con un processo dall'esterno. SIGTERM chiede al processo di terminare in modo ordinato; il processo può ignorarlo o fare cleanup prima di uscire. SIGKILL invece non può essere ignorato: il kernel termina il processo direttamente, senza passare dall'handler.

### trap — handler in bash:

```
trap "echo 'Segnale ricevuto'" SIGTERM
trap "rm -f /tmp/miofile" EXIT # eseguito all'uscita dello script
```

Pseudo-segnali bash: DEBUG, ERR, EXIT, RETURN.

**Nota critica — trap non ereditato dai figli:** gli handler definiti con `trap` appartengono alla shell bash corrente. Quando bash lancia un programma esterno (es. `sleep`), quel processo ha il proprio set di disposizioni dei segnali e non eredita i `trap` della shell padre.

I pseudo-segnali bash (EXIT, DEBUG, ERR, RETURN) non esistono nemmeno nei processi esterni — sono costrutti interni a bash.

## 4. Comandi di monitoraggio

**ps** — snapshot dei processi in esecuzione:

```
ps                # processi della shell corrente
ps $PID           # processo specifico
ps aux            # tutti i processi del sistema
ps hp $PID        # output senza header, utile negli script
```

**Colonne di ps:**

| Colonna | Significato                                                                               |
|---------|-------------------------------------------------------------------------------------------|
| PID     | Process ID                                                                                |
| TTY     | Terminale di controllo (pts/0 = pseudo-terminal, ? = nessun terminale, tipico dei demoni) |
| STAT    | Stato del processo                                                                        |
| TIME    | Tempo CPU totale consumato                                                                |
| CMD/CMD | Comando con argomenti                                                                     |

**Chi restituisce i codici di stato:** è il kernel che mantiene lo stato di ogni processo. ps interroga /proc/<PID>/stat per leggere lo stato corrente.

**Codici di stato (STAT):**

| Codice | Significato                                           |
|--------|-------------------------------------------------------|
| S      | Sleeping (in attesa di I/O o evento)                  |
| R      | Running (in esecuzione o pronta a girare)             |
| T      | Stopped (sospeso da SIGSTOP/SIGTSTP)                  |
| Z      | Zombie (terminato ma non ancora “raccolto” dal padre) |
| N      | Nice positivo (priorità più bassa del normale)        |
| s      | Session leader                                        |
| +      | Foreground                                            |

**ps aux spiegato:** -a = mostra processi di tutti gli utenti; -u = formato utente-oriented (aggiunge USER, %CPU, %MEM, VSZ, RSS); -x = include processi senza terminale di controllo (demoni di sistema). Insieme danno un quadro completo di tutto ciò che gira sulla macchina.

**jobs** — lista dei job della shell corrente:

```
jobs          # mostra [job_id] stato comando
jobs -l       # aggiunge il PID
```

**Processo vs job:** un *processo* è un'entità del kernel identificata dal PID, indipendente dalla shell. Un *job* è un'astrazione della shell per gestire i processi che ha lanciato direttamente. Un job può comprendere più processi in pipeline (cmd1 | cmd2 è un job con due processi). Quando chiudi la shell i job vengono notificati, i processi no.

**fg** — riporta un job in foreground:

```
fg %1         # riporta il job 1 in foreground
```

fg è l'inverso di bg, non di &. & lancia direttamente in background. fg può riprendere sia job stopped (sospesi con Ctrl+Z) che job già in running background.

**watch** — esegue un comando periodicamente:

```
watch "ps aux | grep sleep | grep -v grep"
watch -n 5 "ps aux | grep sleep | grep -v grep" # aggiorna ogni 5 secondi
```

**Come è costruito il comando watch:** ps aux lista tutti i processi → | grep sleep filtra le righe che contengono "sleep" → | grep -v grep esclude la riga del processo grep stesso (che avrebbe "grep sleep" nella colonna CMD e verrebbe inclusa falsamente). È un pattern classico.

**Cambiare il tempo di aggiornamento:** watch -n SECONDI "comando". Il default è 2 secondi. -n 0.5 è il minimo pratico.

---

## 5. Sincronizzazione: wait

wait blocca l'esecuzione dello script fino al completamento dei job in background.

```
sleep 30 &
sleep 40 &
wait      # aspetta entrambi
echo "Tutti terminati"
```

Oppure per un job specifico:

```
wait $PID # aspetta solo quel PID
```

**Interazione con trap:**

**Come funziona trap + wait:** se durante l'attesa di wait arriva un segnale con handler definito via trap, bash non aspetta la fine del wait per eseguire l'handler — esce immediatamente con exit code > 128 (128 + numero segnale), esegue l'handler, e poi il codice riprende *dopo* il wait. Questo è fondamentale per gestire interruzioni pulite: puoi fare cleanup (cancellare file temporanei, ecc.) anche se il programma viene interrotto a metà.

```
trap "echo 'Interrotto!'; rm -f /tmp/mio_output; exit 1" SIGINT
processo_lungo &
wait $!      # se arriva Ctrl+C, esce da wait, esegue trap, poi exit 1
```

---

## 6. Modificatori per processi in background

**nohup** — immunità all'hangup:

Normalmente alla chiusura della shell il kernel invia SIGHUP a tutti i job in background, terminandoli. nohup evita questo:

```
nohup sleep 1000 &          # sopravvive alla chiusura della shell
nohup sleep 1000 > /dev/null 2>&1 & # stesso, ma senza creare nohup.out
```

**Come evitare nohup.out:** di default nohup redirige stdout su nohup.out (nella directory corrente) perché non avrebbe altrove dove mandare l'output. Se non ti serve, redirect esplicito: > /dev/null 2>&1. Il redirect esplicito ha precedenza su quello di nohup.

**disown** — rimuove un job dalla job table:

```
sleep 1000 & disown          # lancia e rimuove subito dalla job table
disown %1                    # rimuove il job 1 già in background
disown -h %1                 # aggiunge immunità a SIGHUP senza rimuovere dalla job table
```

**Disown vs nohup — differenza reale:** la differenza è temporale e di meccanismo. nohup si specifica *all'avvio* del processo — immunizza il processo intercettando SIGHUP prima ancora che parta. disown invece agisce su un processo *già in esecuzione* — rimuove il job dalla job table della shell, così quando la shell chiude non sa più che quel processo esiste e non gli manda SIGHUP. Risultato finale simile (processo sopravvive al logout), meccanismo diverso. disown è utile quando hai già lanciato qualcosa senza nohup e ti ricordi solo dopo che vuoi farlo sopravvivere.

**nice** — priorità di scheduling:

```
nice -n 10 comando          # niceness +10 (priorità più bassa)
nice -n -5 comando          # niceness -5 (priorità più alta, solo root)
```

```
nice nohup long_calc &      # combinazione comune
```

Valori: da -20 (massima priorità) a +19 (minima). Solo root può usare valori negativi.

**Combinazione nice nohup long\_calc & spiegata:** tre modificatori insieme per un calcolo lungo. nice lo lancia a bassa priorità (non rallenta il sistema). nohup lo rende immune a SIGHUP (non muore al logout). & lo mette in background (non blocca il terminale). È il pattern standard per lavori batch che devono girare autonomamente senza impattare il sistema.

---

## 7. Array bash per tracciare processi

In bash gli array possono avere **indici non consecutivi**. Il pattern usa il PID come indice:

```
COMANDO1="sleep 10"
COMANDO2="sleep 20"
```

```
$COMANDO1 & PID[$!]="${COMANDO1}"    # lancia COMANDO1, salva il suo PID come indice
$COMANDO2 & PID[$!]="${COMANDO2}"    # lancia COMANDO2, salva il suo PID come indice
```

```
echo ${!PID[@]}      # stampa gli indici = i PID (es. 1390 1393)
echo ${PID[@]}       # stampa i valori = i comandi (es. sleep 10 sleep 20)
echo ${PID[1390]}    # recupera il comando associato al PID 1390
```

**Cos'è un indice di array:** l'indice è la "chiave" con cui accedi a un elemento. In arr[0]="ciao", 0 è l'indice e "ciao" è il valore. Normalmente gli indici sono numeri consecutivi (0, 1, 2...). Bash però permette indici *arbitrari* — puoi avere arr[1390] e arr[1393] senza riempire i buchi.

**A cosa serve qui:** crei una struttura PID[<pid>] = <comando>. Quando un processo termina e vuoi sapere cosa stava facendo, cerchi \${PID[\$!]}. È una mappa PID → descrizione del lavoro. Utile in script che gestiscono molti processi paralleli.

---

## 8. File temporanei con mktemp

```
FA=$(mktemp)      # crea /tmp/tmp.XXXXXX e restituisce il path
FB=$(mktemp)
# processi paralleli scrivono su FA e FB separatamente
rm -f "$FA" "$FB" # pulizia obbligatoria
```

mktemp risolve il problema della **race condition**: se due processi creassero /tmp/output.txt contemporaneamente con lo stesso nome, uno sovrascriverebbe l'altro. mktemp genera un nome univoco garantito.

---

## Comandi di riferimento

| Comando | Sintassi                 | Descrizione                                               |
|---------|--------------------------|-----------------------------------------------------------|
| &       | comando &                | Lancia in background                                      |
| \$!     | echo \$!                 | PID dell'ultimo processo in background                    |
| jobs    | jobs [-l]                | Lista job della shell corrente (-l aggiunge PID)          |
| fg      | fg %job_id               | Riporta in foreground                                     |
| bg      | bg %job_id               | Manda/riavvia in background (invia SIGCONT)               |
| kill    | kill [-segnale] PID      | Invia segnale a processo                                  |
| ps      | ps [aux\ \$PID]          | Mostra processi (aux = tutti, \$PID = uno specifico)      |
| wait    | wait [PID]               | Aspetta terminazione job (tutti se senza argomenti)       |
| watch   | watch [-n N] "cmd"       | Esegue cmd ogni N secondi (default 2)                     |
| nohup   | nohup cmd &              | Lancia immune a SIGHUP                                    |
| disown  | disown [-h] [%job]       | Rimuove job dalla job table (-h aggiunge immunità SIGHUP) |
| nice    | nice [-n N] cmd          | Lancia con niceness N (-20 max, +19 min)                  |
| trap    | trap "codice"<br>SEGNALE | Definisce handler per segnale                             |
| mktemp  | FA=\$(mktemp)            | Crea file temporaneo sicuro                               |

---

## Esercizi sulla VM

### Esercizio 1 — Processi in background e segnali □

```
sleep 1000 &  
# [1] 12516
```

```
A=$!
```



```
echo $A
# 12516
```

```
ps $A
# PID TTY STAT TIME COMMAND
# 12516 pts/2 SN 0:00 sleep 1000
```

**Cosa leggere da ps \$A:** pts/2 = pseudo-terminal 2 (il tuo terminale SSH). S = sleeping (il processo è in attesa — normale per sleep). N = nice positivo (in realtà qui la shell ha una niceness leggermente più alta). TIME = tempo CPU consumato (quasi zero perché sleep non fa calcoli).

```
jobs
# [1]+  running sleep 1000
```

```
jobs -l
# [1]+ 12516 running sleep 1000
```

**jobs -l vs jobs:** -l aggiunge il PID. Altre opzioni utili: -n mostra solo i job che hanno cambiato stato dall'ultima notifica.

```
kill -STOP $A
# [1]+ 12516 suspended (signal) sleep 1000
```

```
ps $A
# PID TTY STAT TIME COMMAND
# 12516 pts/2 TN 0:00 sleep 1000
```

**Da SN a TN:** S (sleeping) → T (stopped/sospeso). N rimane perché la niceness non è cambiata.

### Bug di Es. 1 — SIGTERM su processo stopped:

```
kill $A      # invia SIGTERM
ps $A        # sembra ancora attivo!
kill -TERM $A # stesso risultato
```

Questo **non è un errore della VM** — è comportamento corretto. Un processo in stato T (stopped) **non può processare segnali** finché non viene ripreso (eccetto SIGKILL, che bypassa tutto). SIGTERM è stato recapitato ma il processo non può gestirlo. Soluzioni:

```
# Opzione 1: SIGKILL (termina forzatamente anche i processi stopped)
kill -KILL $A
```

```
# Opzione 2: riprendi prima, poi termina
kill -CONT $A && kill $A
```

## Esercizio 2 — nohup e disown □

```
sleep 1000 &
sleep 1000 & disown
nohup sleep 1000 & # errore di battitura → "No such file or directory" → nes.
nohup sleep 1000 &
jobs
# [1]- Running  sleep 1000 &
# [2]+ Running  nohup sleep 1000 &
exit
# logout

# Nuova sessione (vagrant ssh)
ps
# PID TTY TIME CMD
# 1270 pts/1 00:00:00 bash
# 1274 pts/1 00:00:00 ps
```

**Perché ps da solo non mostra i sleep:** ps senza argomenti mostra solo i processi della shell corrente. Dopo exit e vagrant ssh sei in una *nuova* shell — i processi sleep girano sotto un'altra sessione terminale (o senza terminale, TTY = ?). Per vederli servono ps aux o ps ax.

```
ps aux | grep sleep | grep -v grep
# vagrant 1249 ... ? S ... sleep 1000
# vagrant 1250 ... ? S ... sleep 1000
# vagrant 1258 ... ? S ... sleep 1000
```

**Perché 3 processi invece di 2:** ti aspettavi che solo 2 sopravvivessero (disown + nohup) e il primo morisse con SIGHUP. In realtà bash non invia SIGHUP ai job in background in tutti i contesti — lo fa solo se l'opzione huponexit è attiva (verificabile con shopt huponexit; di default è off su Debian/Vagrant). Quindi anche il primo sleep 1000 & è sopravvissuto, portando il totale a 3.

---

## Esercizio 3 — wait e watch □

```
# Terminale 1 □
sleep 30 &
sleep 40 &
wait
echo "tutti terminati"
# [1]- Done  sleep 30
# [2]+ Done  sleep 40
```

```
# tutti terminati
```

Il comportamento è corretto: wait aspetta entrambi, poi il prompt ritorna.

```
# Terminale 2 □
```

```
watch "ps auxw | grep sleep | grep -v grep"
```

```
# Error opening terminal: xterm-kitty
```

**Errore xterm-kitty:** il tuo emulatore di terminale locale è Kitty, che usa TERM=xterm-kitty. La VM Debian non ha la definizione del terminale kitty nel database terminfo, quindi watch (che usa ncurses per il refresh dello schermo) non sa come disegnare il terminale. Soluzione: sovrascrivere \$TERM per quella sessione SSH:

```
TERM=xterm-256color watch "ps auxw | grep sleep | grep -v grep"
```

Oppure aggiungi export TERM=xterm-256color nel .bashrc della VM per non doverlo ripetere.

---

## Esercizio 4 — Array con PID □

Annotazione riga per riga di quello che è successo sulla VM:

```
COMANDO1="sleep 10"      # variabile stringa contenente il comando
COMANDO2="sleep 20"
```

```
$COMANDO1 &              # bash espande $COMANDO1 → esegue "sleep 10 &"
                          # poi: PID[$!]="$COMANDO1" → PID[<pid1>]="sleep 10"
# [1] 1390                ← job 1, PID 1390
```

```
$COMANDO2 &              # esegue "sleep 20 &"
                          # poi: PID[$!]="$COMANDO2" → PID[1393]="sleep 20"
# [2] 1393                ← job 2, PID 1393
# [1] Done $COMANDO1      ← sleep 10 è già terminato (era breve)
```

```
echo ${!PID[@]}          # ${!array[@]} = stampa gli INDICI dell'array
# 1390 1393              ← i PID usati come indici
```

```
echo ${PID[@]}           # ${array[@]} = stampa i VALORI
# sleep 10 sleep 20
```

```
echo ${PID[1390]}        # accesso diretto all'elemento con indice 1390
# sleep 10                ← il comando che girava con quel PID
# [2]+ Done $COMANDO2    ← sleep 20 termina
```

---

## Esercizio 5 — Comandi paralleli sincronizzati □ da fare

Consegna: script che accetta due directory come argomenti, conta i file in ciascuna **in parallelo**, aspetta con `wait`, e stampa quale ne contiene di più. Requisiti: controllo argomenti, `&`, `wait`, `mktemp`, pulizia finale.

Prima di vedere la soluzione del prof, prova a implementare lo script da solo. La soluzione usa funzioni `testd()` e `conta()` per separare le responsabilità.

---

## Connessioni

**Con 3A (systemd):** `systemd` è un gestore di processi strutturato — fa a livello di servizi quello che facciamo a mano con `kill`, `wait` e segnali.

**Con Security (S1, S5):** `ps aux` è uno dei primi passi del reconnaissance su un host compromesso. `nohup cmd &` è il pattern per installare backdoor persistenti che sopravvivono al logout.